

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



BÁO CÁO THỰC TẬP

NGÀNH: CÔNG NGHỆ THÔNG TIN

ĐỀ TÀI: XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG
QUẢN LÝ TIẾN ĐỘ ĐỌC, CHIA SẺ VÀ ĐỀ XUẤT
SÁCH ỨNG DỤNG MLOPS VÀ DEVSECOPS

Cán bộ hướng dẫn: Nguyễn Quốc Thắng

Giảng viên đánh giá: ThS. Ngô Xuân
Trường

Sinh viên: Nguyễn Quang Dũng

Mã sinh viên: 23021497

Lớp: K68-CS3

Hà Nội, tháng 10 năm 2026

MỤC LỤC

LỜI CẢM ƠN	ii
DANH MỤC HÌNH	iii
I. GIỚI THIỆU CHUNG	5
II. YÊU CẦU BÀI TOÁN	6
III. TÓM TẮT LÝ THUYẾT, GIẢI PHÁP, THUẬT TOÁN	8
IV. CÁC CÔNG CỤ ĐƯỢC ỨNG DỤNG	14
V. KẾT QUẢ ĐẠT ĐƯỢC VÀ HƯỚNG PHÁT TRIỂN TIẾP THEO	16
TÀI LIỆU THAM KHẢO	19

LỜI CẢM ƠN

Trong quá trình thực tập và thực hiện báo cáo với đề tài này, em đã nhận được sự quan tâm, hướng dẫn và hỗ trợ quý báu từ nhà trường, đơn vị thực tập cùng các thầy cô, anh chị hướng dẫn.

Trước hết, em xin chân thành cảm ơn Ban lãnh đạo và các anh chị tại JITS Innovation Labs đã tạo điều kiện để em được tiếp cận môi trường làm việc thực tế, củng cố kiến thức chuyên môn và rèn luyện kỹ năng nghề nghiệp. Đặc biệt, em xin bày tỏ lòng biết ơn sâu sắc tới cán bộ hướng dẫn Nguyễn Quốc Thắng đã tận tình định hướng, góp ý và hỗ trợ em trong suốt quá trình thực hiện đề tài.

Em cũng xin trân trọng cảm ơn các thầy cô Trường Đại học Công nghệ, Đại học Quốc gia Hà Nội đã trang bị cho em nền tảng kiến thức cần thiết. Em xin cảm ơn ThS. Ngô Xuân Trường đã dành thời gian đánh giá và góp ý cho báo cáo.

Mặc dù đã nỗ lực hoàn thiện, báo cáo khó tránh khỏi những thiếu sót do kinh nghiệm thực tế còn hạn chế. Em rất mong nhận được những ý kiến đóng góp để tiếp tục hoàn thiện kiến thức và kỹ năng trong thời gian tới.

Em xin trân trọng cảm ơn.

DANH MỤC HÌNH

1	Frontend Philobiblus được phát hành trên GitHub Pages	10
2	Các workload và object chính trong cụm GKE của Philobiblus	11
3	Workload Philobiblus và MLOps quan sát trên GKE Console	11
4	Luồng triển khai và các resource Google Cloud của Philobiblus	12
5	Grafana ghi nhận backend trong một lần kiểm tra tải và HPA	15
6	Các lớp kiểm soát từ thay đổi mã nguồn đến workload GKE	16
7	Độ trễ và tỷ lệ lỗi tại các mốc tải chung trước và sau tối ưu	18

I. GIỚI THIỆU CHUNG

a. Giới thiệu Công ty TNHH JITS Innovation Labs

Công ty TNHH JITS Innovation Labs là doanh nghiệp hoạt động trong lĩnh vực phát triển phần mềm và cung cấp dịch vụ công nghệ thông tin tại Hà Nội. Công ty bắt đầu hoạt động từ tháng 6 năm 2016, có quy mô từ 11 đến 50 nhân sự, đặt trụ sở tại Hà Nội. Quy mô này tạo điều kiện để các nhóm kỹ thuật phối hợp trực tiếp, đồng thời yêu cầu mỗi thành viên chủ động trong quá trình nghiên cứu, phát triển và triển khai sản phẩm.

Lĩnh vực chuyên môn của JITS Innovation Labs là các giải pháp công nghệ cho ngân hàng và tổ chức tài chính, bao gồm nền tảng ngân hàng số, hệ thống điểm bán hàng, xử lý dữ liệu theo luồng và tích hợp hệ thống. Đội ngũ của công ty có kinh nghiệm triển khai, tùy biến hệ thống ngân hàng lõi cho khách hàng doanh nghiệp, hỗ trợ các ngân hàng và tổ chức tài chính vì mô cải tiến sản phẩm và thực hiện chuyển đổi số. Đặc thù của các hệ thống tài chính đòi hỏi phần mềm có tính ổn định, khả năng mở rộng, bảo mật và giám sát tốt; vì vậy, các phương pháp DevOps và DevSecOps có vai trò thiết thực trong hoạt động phát triển và vận hành sản phẩm.

Bên cạnh hoạt động phát triển sản phẩm, doanh nghiệp tổ chức chương trình thực tập theo hướng đào tạo gắn với thực hành. Tham gia vào kỳ thực tập năm nay, sinh viên chúng em được tiếp cận kiến thức theo lộ trình, hoàn thành các bài tập nhỏ cho từng chủ đề, lưu trữ sản phẩm trên hệ thống quản lý mã nguồn và trình bày kết quả định kỳ. Hình thức này giúp chúng em hình thành thói quen tự học, viết tài liệu, kiểm chứng kết quả và chịu trách nhiệm đối với sản phẩm đã thực hiện. Trong kỳ thực tập, em tham gia chương trình đào tạo DevOps và lựa chọn hướng chuyên sâu MLOps và Security để vận dụng kiến thức vào dự án tổng hợp.

Thông tin doanh nghiệp được đối chiếu từ trang chủ, trang LinkedIn và thông tin đăng ký doanh nghiệp công khai [1, 2, 3].

b. Giới thiệu công việc thực tập

Trong thời gian thực tập tại JITS Innovation Labs, em đảm nhiệm vị trí DevOps Intern. Em vận dụng kiến thức về Linux, Git, Docker, CI/CD, Kubernetes, Terraform, MLOps và DevSecOps để hoàn thành các bài thực hành và dự án tổng hợp. Quá trình này giúp em hình thành cách tiếp cận công việc từ tổ chức mã nguồn, xây dựng môi trường chạy đến kiểm tra và vận hành sản phẩm.

Từ các kiến thức nền tảng, em xây dựng và đóng gói các ứng dụng web nhiều thành phần, triển khai trên Kubernetes, quản lý hạ tầng bằng Terraform theo mô hình Infrastructure as Code (IaC), kết hợp với kiểm thử tự động và cơ chế CI/CD. Em cũng ứng dụng thêm MLOps và bảo mật vào việc pipeline xử lý dữ liệu, huấn luyện, đánh giá và triển khai mô hình.

Những kinh nghiệm trên được em ứng dụng để phát triển dự án cá nhân Philobiblus, một ứng dụng web hỗ trợ quản lý tiến độ đọc, nhận xét, chia sẻ và gợi ý sách. Sản phẩm kết hợp các nội dung đã thực hành để tổ chức mã nguồn, đóng gói ứng dụng, tự động kiểm thử và triển khai thử nghiệm. Qua đó, em có thể hoàn thiện một sản phẩm từ giai đoạn phát triển đến môi trường vận hành và đánh giá được các yêu cầu về tính ổn định, khả năng mở rộng và bảo mật.

c. Giới thiệu bài toán

Trọng tâm của bài toán là kiểm chứng cách tổ chức và vận hành một ứng dụng web nhiều thành phần: hệ thống phải ổn định khi frontend, backend, cơ sở dữ liệu và model học máy hoạt động đồng thời; bảo vệ dữ liệu theo quyền sở hữu và phạm vi truy cập; cô lập và khôi phục thành phần gặp lỗi; có khả năng autoscale khi tải và số người dùng đồng thời biến động. Philobiblus được em phát triển như là một sản phẩm mẫu để kiểm chứng các yêu cầu trên, có thể hỗ trợ quản lý thư viện và hoạt động đọc, gồm cập nhật tiến độ, nhận xét, chia sẻ và gợi ý sách. Hệ thống được triển khai trên Kubernetes, quản lý hạ tầng bằng Terraform, kết hợp với CI/CD, MLOps và DevSecOps.

II. YÊU CẦU BÀI TOÁN

a. Bối cảnh và mô tả bài toán

Hoạt động đọc thường tạo ra nhiều loại dữ liệu như thư viện cá nhân, tiến độ, nhận xét và lịch sử tương tác. Khi các dữ liệu này được ghi chép hoặc lưu trữ ở nhiều công cụ riêng lẻ, người dùng khó theo dõi quá trình đọc và kiểm soát nội dung đã chia sẻ. Vì vậy, Philobiblus được xây dựng như một hệ thống mẫu tập trung các thao tác quản lý thư viện, cập nhật tiến độ, nhận xét, chia sẻ và gợi ý sách.

Nếu mở rộng cho nhiều người dùng, hệ thống không chỉ cần đáp ứng các chức năng nghiệp vụ mà còn phải xử lý đồng thời nhiều request, phân tách dữ liệu theo tài khoản và kiểm soát phạm vi truy cập. Các thành phần frontend, backend, cơ sở dữ liệu và recommendation service được tách riêng để giới hạn ảnh hưởng khi có lỗi, triển khai độc lập và mở rộng theo tải. Đây là cơ sở để bài toán được triển khai trên Kubernetes với các yêu cầu về bảo mật, khả năng phục hồi, autoscale và theo dõi vận hành.

b. Mục tiêu của hệ thống

Mục tiêu nghiệp vụ của Philobiblus là hoàn thiện các chức năng cơ bản để quản lý thư viện, tiến độ đọc, nhận xét, chia sẻ và gợi ý sách; dữ liệu phải được gắn với đúng tài khoản và phạm vi truy cập.

Mục tiêu kỹ thuật là triển khai hệ thống nhiều thành phần trên GKE Autopilot bằng Terraform và Helm; xây dựng CI/CD để kiểm thử, quét bảo mật, build và phát hành image; quản lý snapshot, run, artifact và model version bằng DVC và MLflow. Phần vận hành cần kiểm chứng khả năng cô lập và khôi phục dịch vụ, autoscale theo tải và quan sát workload qua metric, log và trạng thái rollout.

c. Yêu cầu của hệ thống

Yêu cầu chức năng

Quản lý tài khoản và quyền truy cập. Người dùng có thể đăng ký, đăng nhập, cập nhật hồ sơ và đăng xuất; backend phải xác thực token, kiểm tra chủ sở hữu tài nguyên và áp dụng quyền quản trị ở phía máy chủ.

Quản lý thư viện và tiến độ đọc. Người dùng có thể tạo, tìm kiếm, chỉnh sửa và xóa sách thuộc quyền sở hữu; mỗi sách phải lưu được trạng thái dự định đọc, đang đọc hoặc đã hoàn thành, cùng tiến độ và lịch sử thay đổi.

Tương tác và chia sẻ. Người dùng có thể viết nhận xét, chấm điểm, theo dõi tài khoản khác và chia sẻ sách theo các mức public, private hoặc restricted; dữ liệu không được hiển thị vượt quá visibility đã chọn.

Gợi ý nội dung. Hệ thống phải trả về danh sách sách liên quan từ catalog hợp lệ, loại bỏ sách nguồn và các bản ghi không đủ metadata, đồng thời vẫn phục vụ được thư viện khi dịch vụ gợi ý tạm thời không sẵn sàng.

Quản trị vòng đời model. Hệ thống phải tạo snapshot dữ liệu, ghi nhận run và metric, kiểm tra chất lượng candidate rồi chỉ phát hành model đạt điều kiện; model đang phục vụ phải truy nguyên được về snapshot, mã nguồn và artifact tương ứng.

Yêu cầu phi chức năng

Bảo mật và phân quyền. Mật khẩu phải được băm, API phải kiểm tra JWT và quyền sở hữu, secret không đặt trong mã nguồn; các lớp Gateway, Cloud Armor, NetworkPolicy và securityContext phải hạn chế truy cập từ ngoài vào trong cluster.

Khả năng triển khai lặp lại. Hạ tầng GKE, Cloud SQL, mạng và quyền truy cập phải được mô tả bằng Terraform để có thể kiểm tra thay đổi, tái tạo môi trường và tránh cấu hình thủ công không truy vết được. Đây là lý do dự án áp dụng Infrastructure as Code.

Khả năng mở rộng và tự phục hồi. Các thành phần chạy dạng container phải được đóng gói thành Deployment hoặc Job trên Kubernetes, có readiness/liveness probe, giới hạn tài nguyên và cơ chế rollout; nhờ đó workload có thể tự khởi động lại và mở rộng theo nhu cầu.

Tính lặp lại của phát hành. Mỗi thay đổi phải đi qua CI/CD gồm kiểm thử, kiểm tra dependency, quét bảo mật, build image và triển khai bằng Helm với image digest xác định; việc phát hành không phụ thuộc vào thao tác thủ công trên cluster.

Quản lý vòng đời dữ liệu và model. Snapshot, pipeline, metric, artifact và model version phải được lưu vết bằng DVC và MLflow, cho phép tái hiện một lần huấn luyện và rollback khi candidate không đạt quality gate. Đây là yêu cầu cốt lõi dẫn đến việc áp dụng MLOps.

Quan sát và xử lý sự cố. Hệ thống phải xuất metric, log và trạng thái rollout để Cloud Monitoring, Cloud Logging và Managed Service for Prometheus có thể phát hiện lỗi, theo dõi tải và hỗ trợ đối chiếu nguyên nhân.

Khả năng kiểm thử và bảo trì. Backend, recommendation service, Helm chart và Terraform module phải được kiểm thử độc lập; cấu trúc module, schema và hợp đồng API cần đủ rõ để thay đổi một thành phần không làm hỏng các thành phần còn lại.

d. Phạm vi bài toán và trách nhiệm pháp lý liên quan

Philobiblus là dự án độc lập do em thực hiện từ phân tích yêu cầu, phát triển giao diện, API và cơ sở dữ liệu đến MLOps, kiểm thử, đóng gói và triển khai. Sản phẩm được đánh giá qua các luồng quản lý thư viện, tiến độ, nhận xét, chia sẻ, gợi ý và khả năng duy trì ranh giới quyền hạn.

Dự án không xử lý thanh toán, giao nhận hoặc phân phối nội dung số. Hệ thống chỉ lưu trữ và hiển thị các bản ghi mô tả sách do người dùng cung cấp hoặc lấy từ nguồn phù hợp, gồm tên sách,

tác giả, thể loại, thể, số trang và thông tin hoặc đường dẫn ảnh bìa. Hệ thống không lưu trữ, sao chép hoặc cung cấp toàn văn, ebook, audiobook hay bản scan của sách. Việc sử dụng ảnh bìa trong môi trường thực tế phải tuân theo quyền và điều khoản của nguồn cung cấp; báo cáo này chỉ mô tả phạm vi kỹ thuật của hệ thống.

Hệ thống gợi ý sách sử dụng thông tin mô tả của từng cuốn sách trong cơ sở dữ liệu, chẳng hạn như tên sách, tác giả, thể loại và thể. Dữ liệu này được lưu thành snapshot để huấn luyện và đánh giá mô hình. Sau đó, mô hình xác định mức độ tương đồng giữa các cuốn sách và đề xuất những cuốn gần với sách người dùng đang xem hoặc đã chọn, trong phạm vi danh sách sách hiện có. Hệ thống gợi ý sách chỉ phục vụ mục đích kiểm chứng pipeline MLOps, chưa hướng tới xây dựng dịch vụ thương mại hoặc mô hình cá nhân hóa ở quy mô lớn.

III. TÓM TẮT LÝ THUYẾT, GIẢI PHÁP, THUẬT TOÁN

a. Các lý thuyết, giải pháp và thuật toán liên quan

Các lý thuyết nền tảng

DevOps: là phương pháp kết nối hoạt động phát triển và vận hành thông qua một quy trình liên tục gồm viết mã, kiểm thử, đóng gói, triển khai và theo dõi hệ thống. Trong dự án, DevOps được thể hiện qua CI/CD, container, Infrastructure as Code, Helm và Kubernetes để giảm thao tác thủ công và bảo đảm mỗi phiên bản được triển khai theo cùng một quy trình [4].

DevSecOps mở rộng DevOps bằng cách đưa bảo mật vào từng bước của quy trình phát triển, thay vì chỉ kiểm tra trước khi đưa hệ thống lên production. Các nội dung kiểm tra gồm mã nguồn, dependency, container image, secret, cấu hình hạ tầng và quyền truy cập. Với Philobiblus, các bước kiểm tra này được tích hợp vào CI/CD để phát hiện sớm rủi ro trước khi artifact được phát hành [5].

CI/CD tự động hóa chuỗi kiểm thử, đóng gói và phát hành, giúp mỗi thay đổi được kiểm tra trước khi đi vào môi trường chạy [4, 6].

Infrastructure as Code biểu diễn hạ tầng bằng code có thể được quản lý bằng Git, giúp quản lý hạ tầng nhất quán và cho phép xem trước thay đổi trước khi áp dụng [7, 8].

Containerization giúp đóng gói ứng dụng cùng dependency trong image bất biến, giảm khác biệt giữa môi trường phát triển, kiểm thử và production [9].

Điều phối workload: Kubernetes quản lý Pod, Deployment, Service, probe, rollout và autoscaling để workload có thể tự phục hồi và mở rộng khi lượng người dùng biến động [10].

MLOps là tập hợp các thực hành kết hợp giữa phát triển mô hình ML và vận hành hệ thống nhằm tự động hóa, chuẩn hóa và quản lý vòng đời mô hình một cách hiệu quả [11].

Observability: là việc đo lường và theo dõi trạng thái bên trong của một hệ thống phần mềm dựa trên các dữ liệu đầu ra mà nó tạo ra trong quá trình hoạt động [12].

Các giải pháp và công cụ áp dụng

Terraform: là một công cụ giúp quản lý và tự động hóa hạ tầng công nghệ dưới dạng mã nguồn (IaC) thông qua các bước viết cấu hình, lập kế hoạch và áp dụng thay đổi [8].

Google Cloud và GKE: GKE Autopilot cung cấp cluster Kubernetes managed, còn Google Cloud cung cấp các dịch vụ cloud để deploy hệ thống vào hoạt động [13].

Docker tạo image cho các thành phần của hệ thống như frontend, backend, mô hình gợi ý,... để có thể deploy lên các môi trường khác như cloud.

Helm giúp người dùng dễ dàng định nghĩa, cài đặt, nâng cấp và quản lý các cấu hình phức tạp trên cụm Kubernetes thông qua các gói đóng gói sẵn gọi là Chart [14].

GitHub Actions: là nền tảng tự động hóa và CI/CD tích hợp trực tiếp ngay trên GitHub [6].

DVC (Data Version Control) là công cụ quản lý phiên bản dữ liệu và quy trình xử lý dữ liệu, hoạt động cùng với Git. [15].

MLflow là nền tảng theo dõi thí nghiệm và quản lý vòng đời mô hình học máy. Công cụ lưu thông tin về run, parameter, metric, artifact và model version [16].

Cloud SQL là dịch vụ cơ sở dữ liệu quan hệ được quản lý hoàn toàn trên Google Cloud, trong dự án sử dụng PostgreSQL để lưu dữ liệu nghiệp vụ [17].

Redis là máy chủ cấu trúc dữ liệu trong bộ nhớ, thường được dùng làm cache, queue hoặc lớp xử lý dữ liệu có yêu cầu phản hồi nhanh [18].

Cloud Monitoring là dịch vụ của Google Cloud dùng để thu thập, hiển thị và cảnh báo dựa trên các chỉ số vận hành của tài nguyên [19].

Managed Service for Prometheus: Managed Service for Prometheus là dịch vụ quản lý Prometheus trên Google Cloud, dùng để thu thập và truy vấn metric do các workload Kubernetes xuất ra [20, 21].

Các thuật toán sử dụng

TF-IDF biến metadata sách thành vector thưa, trong đó các từ xuất hiện nhiều trong một sách nhưng ít xuất hiện trong catalog có trọng số cao [22].

Cosine similarity là phương pháp đo góc giữa hai vector TF-IDF để xếp hạng các sách có nội dung gần với sách nguồn mà không phụ thuộc độ dài tuyệt đối của chuỗi [22].

Snapshot hash và quality gate: Hệ thống băm nội dung logic của snapshot, đối chiếu schema và mức suy giảm catalog trước khi cho phép candidate thay champion; cách kiểm soát này được triển khai riêng cho pipeline của Philobiblus và liên kết với dữ liệu, run cùng model version [15, 16].

b. Quá trình phát triển hệ thống

Hoàn thiện mã nguồn và artifact

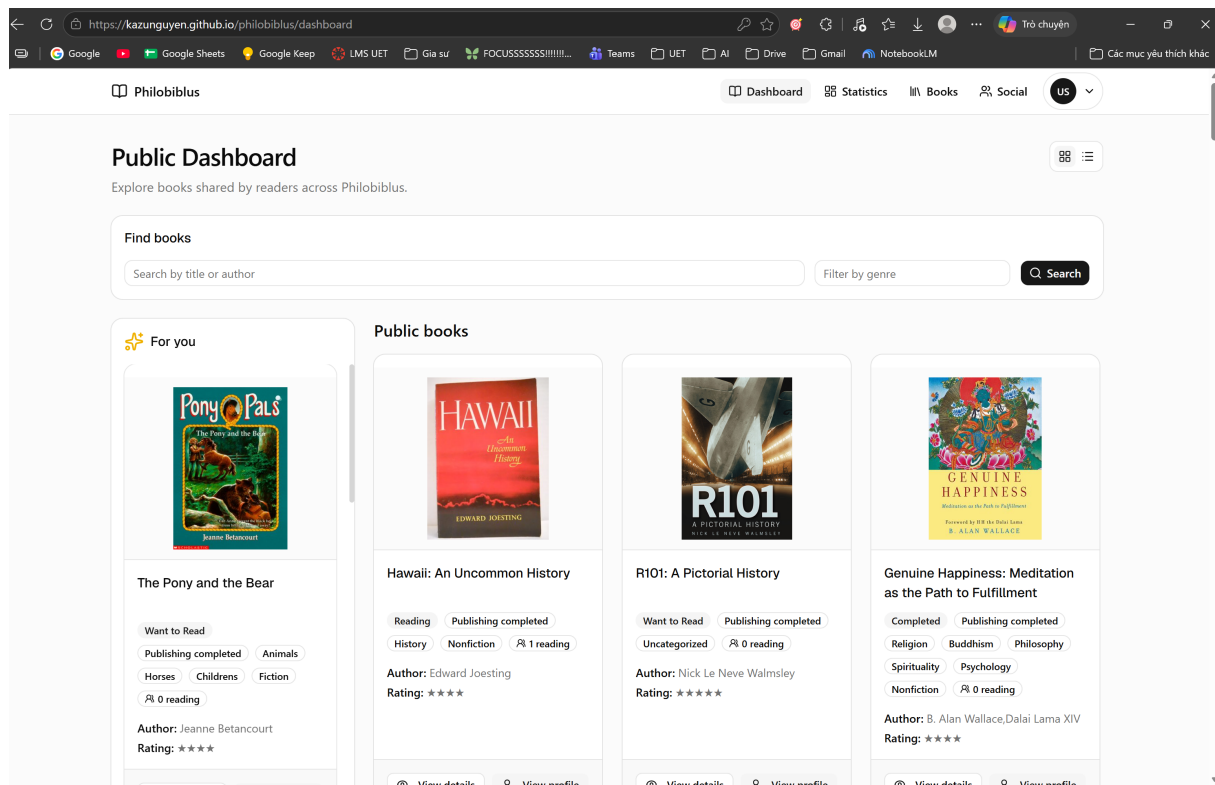
Frontend được phát triển bằng React 18 và Vite. Giao diện tổ chức theo component-based architecture, sử dụng shadcn/ui với style base-nova, Tailwind CSS và Lucide icons; React Router quản lý điều hướng, còn các Context Provider tách trạng thái xác thực, chế độ xem và cài đặt. Các service module gọi REST API, nên frontend không thể truy cập trực tiếp vào database hoặc model artifact. Frontend được deploy trực tiếp lên GitHub Pages.

Backend dùng FastAPI và Uvicorn, tổ chức router theo domain như authentication, books, users,

reviews và reading history. SQLAlchemy quản lý model và session database, Pydantic schema kiểm tra dữ liệu vào ra, còn dependency injection cung cấp database session cho từng request. JWT, quyền sở hữu và visibility đối với thông tin sách được kiểm tra khi gọi API; middleware bổ sung CORS, metric và trace. Cuối cùng, em đóng gói toàn bộ thành docker image để triển khai lên Kubernetes.

Cơ sở dữ liệu và cache: PostgreSQL lưu dữ liệu nghiệp vụ theo schema được khai báo bằng SQLAlchemy; môi trường production sử dụng Cloud SQL thay cho database container trong GKE. Redis đảm nhiệm cache catalog và rate limit, giúp giảm số lần truy vấn database và giới hạn các request tốn tài nguyên. Backend cấu hình connection pool, timeout và cơ chế bỏ qua cache khi Redis tạm thời không khả dụng.

Recommendation service: chạy bằng FastAPI, sử dụng các metadata của sách như tên, tác giả, thể loại để gợi ý sách tương đồng. Pipeline tạo snapshot catalog, chuẩn hóa dữ liệu và dùng TF-IDF để huấn luyện model Joblib; DVC quản lý các bước xử lý, còn MLflow lưu metric và phiên bản model. Khi hoạt động, hệ thống xếp hạng sách bằng cosine similarity và đưa ra top 5 sách có cosine similarity cao nhất.



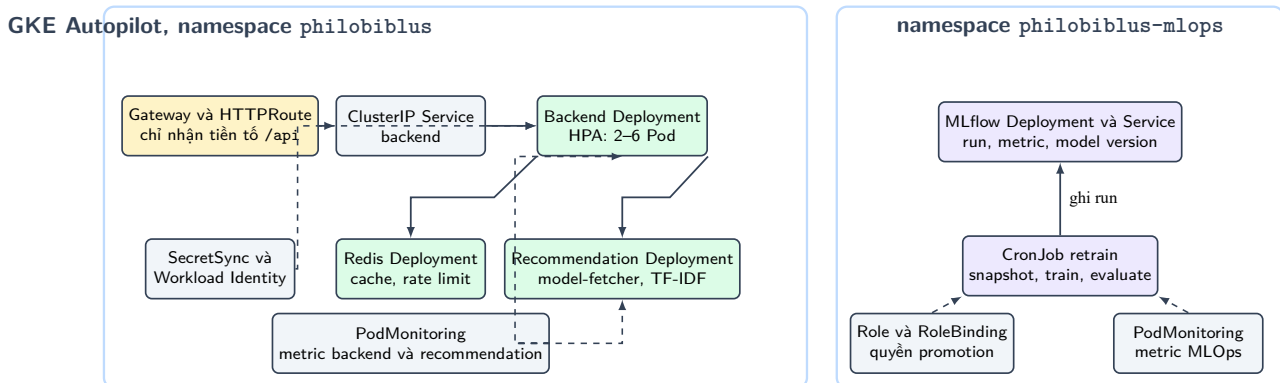
Hình 1: Frontend Philobiblus được phát hành trên GitHub Pages

Đóng gói workload bằng Helm

Sau khi các service hoàn thiện, Helm được dùng để đưa chúng vào cùng một quy trình triển khai trên Kubernetes. Các cấu hình cần thiết được gom thành những gói triển khai có thể kiểm tra, version hóa và cập nhật lặp lại giữa các môi trường. Cách làm này giảm thao tác thủ công, giữ cấu hình nhất quán và cho phép quay lại bản phát hành trước khi cần thiết.

Các thành phần phục vụ người dùng và pipeline MLOps được tách thành hai nhóm có vòng đời

riêng. Nhóm thứ nhất duy trì các chức năng của ứng dụng, còn nhóm thứ hai phụ trách theo dõi thí nghiệm và cập nhật model. Hai nhóm vẫn dùng chung cụm Kubernetes nhưng có thể triển khai độc lập, nên việc huấn luyện hoặc thay đổi model không làm gián đoạn các chức năng chính của hệ thống.



Hình 2: Các workload và object chính trong cụm GKE của Philobiblus

Workloads

Refresh

Deploy

Create Job

Delete

Cluster

Namespace

Reset

Save

Overview

Observability

Cost optimisation

Filter

Is system object : False

Cluster : philobiblus-dev-gke

Filter workloads

☐	Name ↑	Status	Type	Pods	Node type ?	Namespace	Cluster
☐	mlflow	✔ OK	Deployment	1/1	Autopilot-managed	philobiblus-mlops	philobiblus-dev-gke
☐	philobiblus-backend	✔ OK	Deployment	2/2	Autopilot-managed	philobiblus	philobiblus-dev-gke
☐	philobiblus-recommendation	✔ OK	Deployment	1/1	Autopilot-managed	philobiblus	philobiblus-dev-gke
☐	philobiblus-redis	✔ OK	Deployment	1/1	Autopilot-managed	philobiblus	philobiblus-dev-gke
☐	philobiblus-retrain	✔ OK	Cron Job	0/1	Autopilot-managed	philobiblus-mlops	philobiblus-dev-gke
☐	philobiblus-seed	✔ OK	Job	0/1	Autopilot-managed	philobiblus	philobiblus-dev-gke

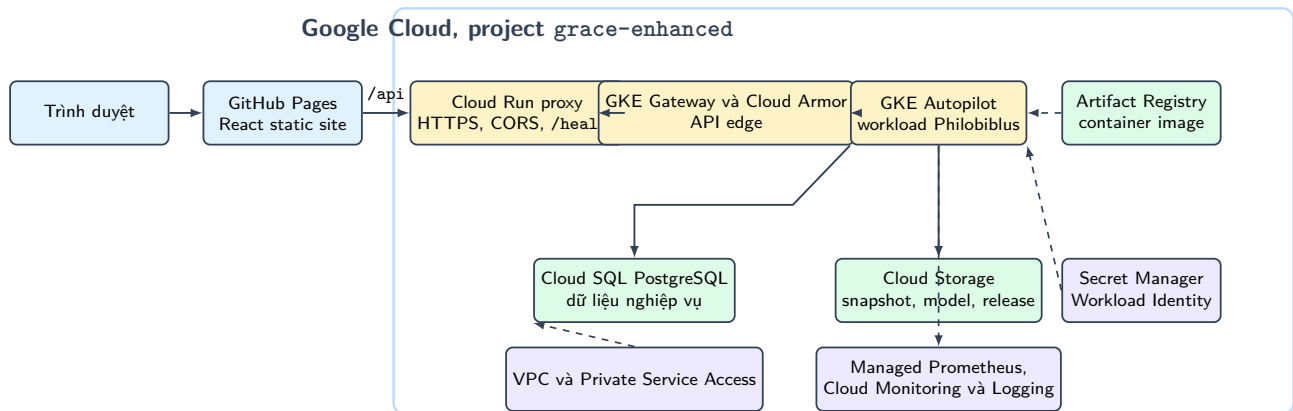
Hình 3: Workload Philobiblus và MLOps quan sát trên GKE Console

Triển khai trên Google Cloud

Terraform chia hạ tầng thành các lớp có phụ thuộc rõ ràng. Lớp foundation tạo VPC, Private Service Access, Artifact Registry, Secret Manager, service account và Cloud SQL PostgreSQL. Lớp GKE platform tạo GKE Autopilot, địa chỉ IP cho Gateway và Cloud Armor. Lớp GKE application cấp namespace, quota, Workload Identity rồi gọi Helm release với external database, Redis, Gateway và Managed Prometheus. Riêng lớp MLOps tạo bucket GCS lưu snapshot, model và release; MLflow và trainer truy cập các tài nguyên này qua service account tương ứng.

GitHub Pages không gọi trực tiếp địa chỉ IP của Gateway. Cloud Run proxy nhận HTTPS request từ frontend, cung cấp health endpoint riêng và chuyển tiếp /api tới Gateway. Điều này giúp frontend dùng một URL HTTPS ổn định khi Gateway chưa gắn custom domain và certificate riêng. Tại thời điểm kiểm tra bằng gcloud ngày 04/10/2026, cluster philobiblus-dev-gke có trạng thái RUNNING, Cloud Run service philobiblus-dev-proxy có trạng thái READY, Cloud

SQL instance philobiblus-dev-postgres có trạng thái RUNNABLE, Artifact Registry và bucket grace-enhanced-philobiblus-mlops đều tồn tại.



Hình 4: Luồng triển khai và các resource Google Cloud của Philobiblus

Kiểm thử và xác minh

Các ca kiểm thử được thực hiện trên trạng thái source hiện tại ngày 04/10/2026. Backend và ML chạy trong môi trường Python tạm với dependency lấy từ file requirements; build frontend dùng fixture tương đương GitHub Pages; Helm dùng fixture không chứa secret như workflow CI.

Mã	Ca kiểm thử và phạm vi	Kết quả
TC-01	Backend pytest: API nghiệp vụ, JWT và phân quyền, cache catalog, rate limit, observability, recommendation và load shedding.	63/63 đạt trong 60,03 giây; 2 cảnh báo deprecation từ dependency.
TC-02	ML pytest: recommendation service, TF-IDF recommender, quality gate và snapshot validation.	17/17 đạt trong 6,89 giây; 1 cảnh báo deprecation từ Starlette.
TC-03	Frontend build Vite với biến môi trường tương đương workflow GitHub Pages.	Thành công: 2.105 module được bundle trong 8,27 giây.
TC-04	Lint và render chart philobiblus và philobiblus-mlops với fixture không chứa secret.	2/2 chart thành công; chỉ có khuyến nghị thêm icon cho Chart.yaml.
TC-05	Truy vấn chỉ đọc trạng thái GKE, Cloud Run, Cloud SQL, Artifact Registry và Cloud Storage.	GKE RUNNING, proxy READY, Cloud SQL RUNNABLE; repository và bucket MLOps tồn tại.

Table 1: Kết quả kiểm thử và xác minh triển khai

Backend pytest dùng SQLite in-memory để cô lập test API, nên không thay thế kiểm thử tích hợp với Cloud SQL. Helm lint và template kiểm tra tính hợp lệ của manifest trước triển khai, không thay thế việc rollout trên cluster. Hai giới hạn này được giữ tách biệt với kết quả xác minh resource GCP để tránh diễn giải quá mức phạm vi kiểm thử.

c. Liên hệ và so sánh với các cách đã nêu

TF-IDF không cần ma trận tương tác dày và xử lý tốt catalog mới, đổi lại phụ thuộc chất lượng metadata. Khi lịch sử đọc đủ lớn, hệ thống có thể kết hợp điểm nội dung với hành vi cộng đồng.

Các đánh đổi kỹ thuật trong giải pháp

Redis cache catalog trong 30 giây và được invalidation khi dữ liệu thay đổi. Nếu Redis lỗi, backend bỏ qua cache để tiếp tục phục vụ, chấp nhận tăng tải tạm thời cho Cloud SQL.

Mỗi backend Pod dùng pool 3 kết nối và tối đa 2 kết nối vượt mức, timeout 10 giây. Giới hạn này được tính cùng HPA để tránh tổng số connection vượt khả năng của Cloud SQL db-f1-micro.

TF-IDF dễ giải thích nhưng chưa khai thác sâu hành vi cộng đồng. GKE chuẩn hóa rollout, probe và autoscaling, đổi lại yêu cầu quản lý chặt tài nguyên, quyền cloud và chi phí.

IV. CÁC CÔNG CỤ ĐƯỢC ỨNG DỤNG

a. Không cần mô tả cách cài đặt các ứng dụng phụ

Người dùng truy cập Philobiblus bằng trình duyệt và không phải cài phần mềm desktop. GKE, Helm, Terraform, Cloud SQL và các dịch vụ quan sát chỉ phục vụ triển khai, vận hành.

b. Mô tả ngắn gọn cách cài đặt phần mềm được phát triển

Frontend tĩnh được phát hành trên GitHub Pages. Origin HTTPS của Pages được khai báo trong CORS của backend; request API đi qua Cloud Run HTTPS proxy rồi mới tới GKE Gateway. Backend và recommendation chạy trong namespace philobiblus, Redis cung cấp cache và điều tiết tải, còn dữ liệu nghiệp vụ nằm trên Cloud SQL.

Quy trình triển khai trên GKE

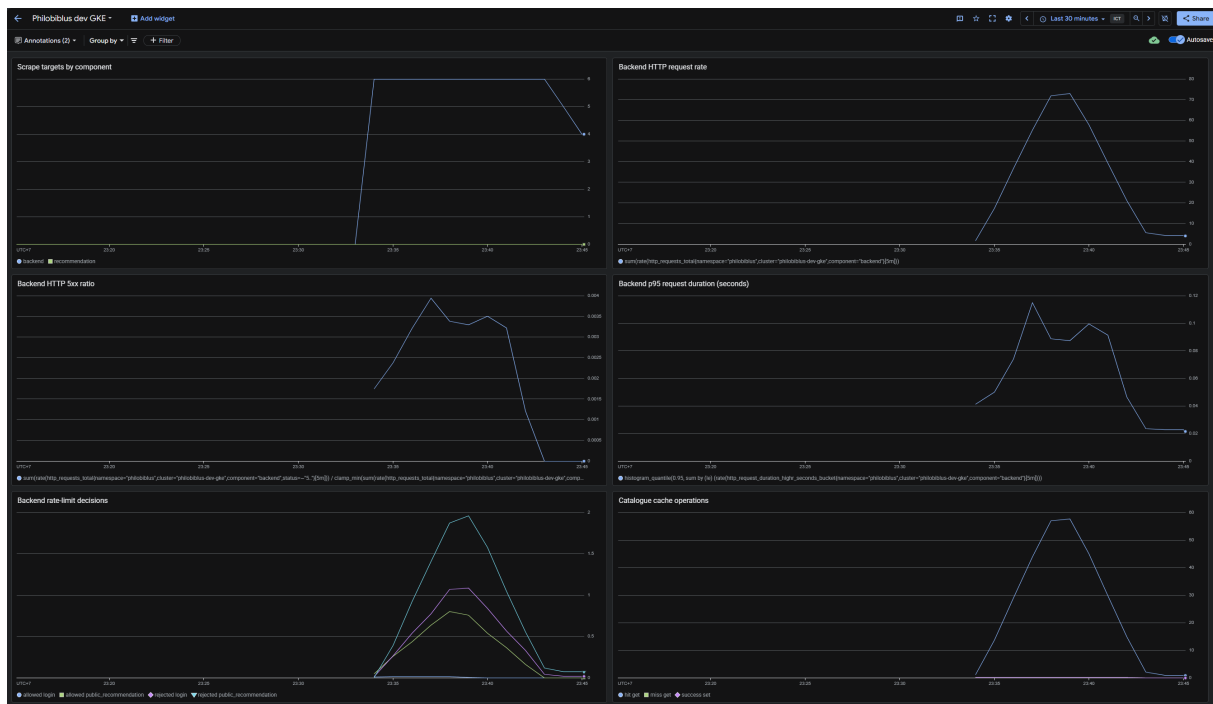
Hạ tầng được tách thành các Terraform state foundation, gke-platform, gke-app, gke-mlops, gke-observability và https-proxy. foundation quản lý VPC, Artifact Registry, Secret Manager, Cloud SQL và các quyền nền; gke-platform quản lý GKE Autopilot, Gateway và Cloud Armor; gke-app quản lý namespace, Workload Identity, Helm release và cấu hình ứng dụng; gke-mlops quản lý bucket versioned, IAM theo prefix, MLflow và pipeline retrain; gke-observability quản lý Cloud Monitoring dashboard cùng alert policy; https-proxy quản lý Cloud Run proxy. Tách state riêng giúp thay đổi dashboard, proxy hoặc model pipeline mà không lập kế hoạch lại toàn bộ cluster.

Image được tham chiếu bằng digest bất biến để cùng một release luôn trở tới cùng nội dung. Helm tạo Deployment, Service, Redis, HTTPRoute, GCPBackendPolicy, NetworkPolicy, PodMonitoring, readiness/liveness probe và HPA. Backend dùng tối thiểu 2, tối đa 6 Pod trong đợt đo tải; probe chỉ đưa Pod vào Service khi sẵn sàng. Workload nhận quyền Google Cloud qua Workload Identity, trong khi secret được cung cấp ngoài chart thay vì ghi trực tiếp trong repository.

Release đi qua kiểm thử, build image, lấy digest, validate Terraform, render Helm và chờ roll-out. Metric nội bộ được PodMonitoring thu vào Google Managed Service for Prometheus; Cloud Monitoring và Cloud Logging là giao diện quan sát của GKE.

Quan sát hệ thống và xử lý sự cố

FastAPI xuất /metrics trong mạng nội bộ. PodMonitoring thu request rate, latency, mã phản hồi, rate limit, cache operation và quyết định load shedding của recommendation; Cloud Monitoring kết hợp chúng với CPU, memory, restart và số replica. Log stdout/stderr được Cloud Logging thu theo cluster, namespace và container.



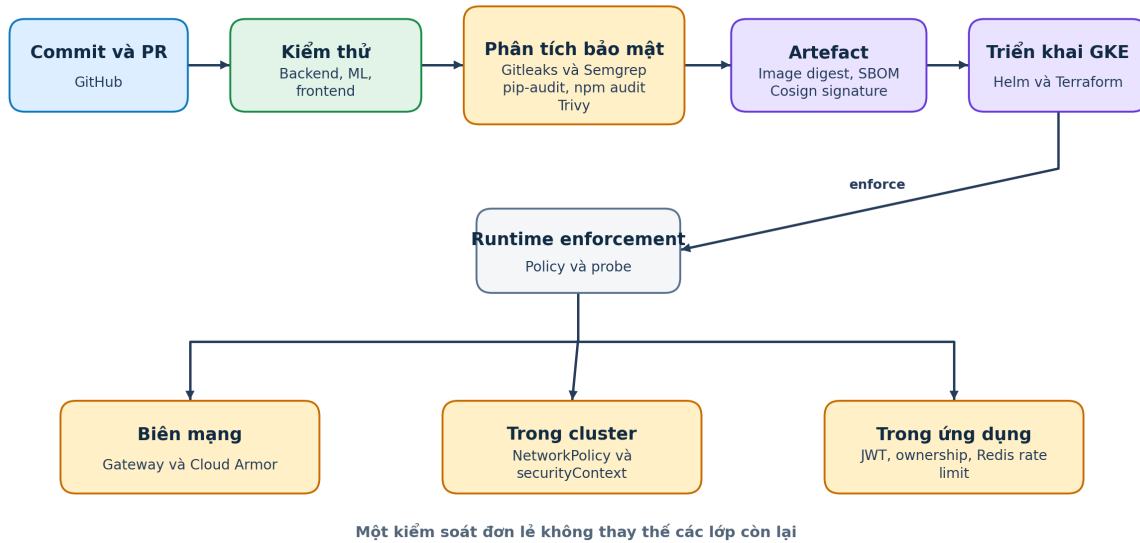
Hình 5: Grafana ghi nhận backend trong một lần kiểm tra tải và HPA

Dashboard ghi lại diễn biến target, request rate và p95 trong lần tạo tải. Khi phân tích sự cố cần đối chiếu khoảng thời gian, metric tài nguyên và log, không kết luận chỉ từ một panel.

Chuỗi kiểm soát DevSecOps

CI sử dụng Gitleaks, Semgrep, pip-audit, npm audit và Trivy; bước phát hành tạo SBOM và ký image bằng Cosign trước khi triển khai. Cảnh báo quét được phân loại theo khả năng khai thác thay vì xử lý như nhau.

Chuỗi kiểm soát DevSecOps từ commit đến GKE



Hình 6: Các lớp kiểm soát từ thay đổi mã nguồn đến workload GKE

Sau khi workload hoạt động, Cloud Armor, Gateway, NetworkPolicy, securityContext, Redis rate limiter và circuit breaker của recommendation bảo vệ các ranh giới khác nhau. HTTPRoute chỉ công khai /api; /metrics, /docs, /openapi.json và /health trả 404 khi gọi trực tiếp qua Gateway nhưng vẫn dùng được cho probe và thu metric nội bộ. Riêng Cloud Run proxy trả 200 tại /health để liveness probe của chính proxy không phụ thuộc vào HTTPRoute. Request thử SQL injection và XSS khớp rule CRS 4.22 của Cloud Armor ở chế độ preview. Chuỗi đăng nhập sai vượt ngưỡng bị rate limiter ứng dụng trả 429, cho thấy lớp bảo vệ trong backend vẫn hoạt động khi rule biên chưa chuyển sang Enforce.

V. KẾT QUẢ ĐẠT ĐƯỢC VÀ HƯỚNG PHÁT TRIỂN TIẾP THEO

a. Kỹ năng và kiến thức thu thập được

Trong kỳ thực tập, em theo dõi toàn bộ hệ thống từ mã nguồn, kiểm thử và image đến Terraform, Helm, GKE và quan sát vận hành. Em hiểu rõ hơn cách Service, probe, resource, secret, policy mạng và rollout phối hợp trong một workload thực tế.

Với MLOps và DevSecOps, em thực hành quản lý snapshot, artifact, quality gate, JWT, quyền sở hữu, Cloud Armor, NetworkPolicy và kiểm tra chuỗi cung ứng phần mềm.

Kết quả kỹ thuật đã kiểm chứng

Đợt này có tổng cộng 63 test backend, 8 test recommendation, frontend production build, lint/render hai Helm chart và terraform validate cho các module GKE. Hai test backend mới kiểm tra circuit breaker recommendation khi vượt ngưỡng traffic và khi Redis không khả dụng. Các kiểm tra này bao phủ logic và cấu hình, chưa thay thế kiểm thử người dùng cuối.

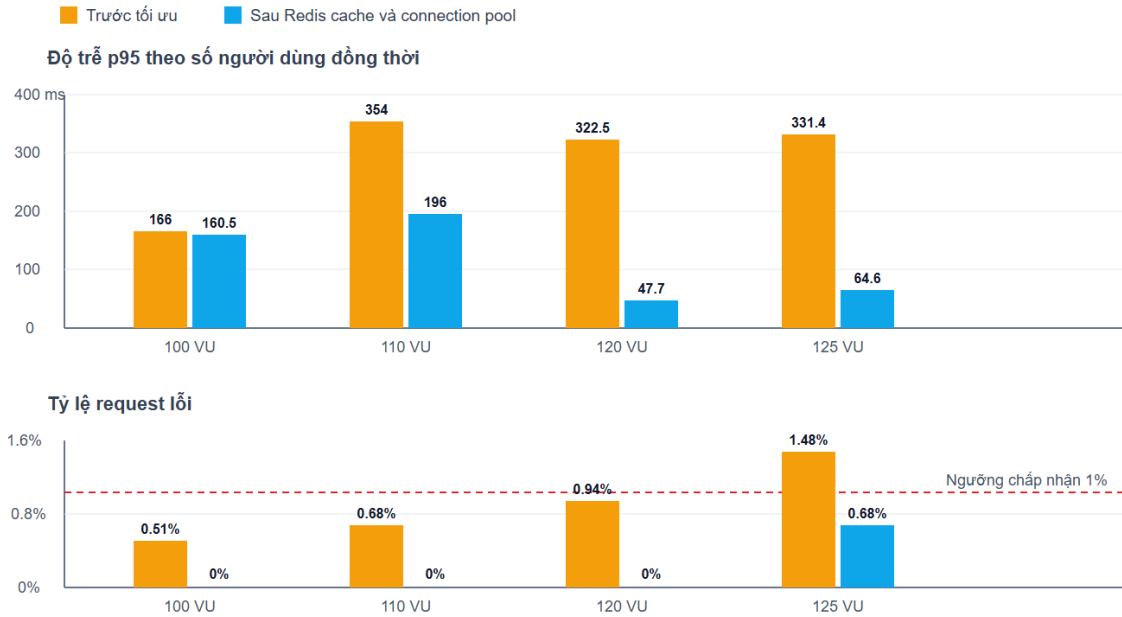
Ngày 24 tháng 9 năm 2026, pipeline trên project grace-enhanced và cluster philobiblus-dev-gke tạo snapshot từ 100 sách, ghi MLflow run và đăng ký model version 1. Candidate insufficient_data không được promote vì thiếu hồ sơ tương tác.

Release hardening triển khai 2 backend Pod, Redis, GCPBackendPolicy và ba NetworkPolicy. Các endpoint vận hành vẫn dùng nội bộ nhưng bị chặn qua Gateway; không ghi nhận OOMKilled, redis_error hoặc cạn connection trong lần xác minh. Ngày 4 tháng 10 năm 2026, alert backend scrape target unavailable được xử lý bằng cách đổi namespace collector trong NetworkPolicy từ gmp-system sang gke-gmp-system; target backend sau đó trở lại trạng thái up.

b. Hướng phát triển tiếp theo để hoàn thiện giải pháp

Lần kiểm tra đầu đạt 120 người dùng đồng thời với 23,4 request/giây, p95 322,5 ms và 0,94 phần trăm lỗi; tại 125 người dùng, lỗi tăng lên 1,48 phần trăm. Sau khi thêm Redis cache TTL 30 giây và giới hạn pool 3 + overflow 2 cho mỗi Pod, mốc 150 người dùng đạt 29,6 request/giây, p95 138,6 ms và không có request lỗi.

So sánh tải trước và sau tối ưu ngày 24 tháng 9 năm 2026



Hình 7: Độ trễ và tỷ lệ lỗi tại các mức tải chung trước và sau tối ưu

150 người dùng là mức cao nhất đã kiểm tra; ở lần benchmark này recommendation service được tắt để cô lập backend cùng database. Vì vậy kết quả không đại diện cho năng lực khi đồng thời phục vụ workload gợi ý. Môi trường dev hiện bật recommendation service và dùng circuit breaker Redis chung: vượt 12 request trong 10 giây thì API trả unavailable trong 30 giây, sau đó tự phục hồi. Mức vận hành đề xuất vẫn là 120 đến 130 người dùng đồng thời, khoảng 24 request/giây, trước khi thực hiện lần đo đầy đủ hơn có recommendation service.

Cloud Armor vẫn ở chế độ preview để theo dõi false positive. Bước tiếp theo là xác minh chữ ký image, trace, PolicyReport và notification trước khi chuyển từng rule sang Enforce.

Hướng phát triển ưu tiên thu thập đủ hồ sơ đọc cho holdout evaluation, sau đó thử mô hình lại và canary với hai release stable, candidate độc lập.

TÀI LIỆU THAM KHẢO

- [1] JITS Innovation Labs. (2026) Jits innovation labs. [Online]. Available: <https://jitsinnovationlabs.com/>
- [2] ——. (2026) Jits innovation labs trên linkedin. [Online]. Available: <https://www.linkedin.com/company/jits-innovation-labs>
- [3] Cổng thông tin quốc gia về đăng ký doanh nghiệp. (2026) Thông tin đăng ký doanh nghiệp công khai. [Online]. Available: <https://dangkykinhdoanh.gov.vn/>
- [4] Amazon Web Services. (2026) What is devops? [Online]. Available: <https://aws.amazon.com/devops/what-is-devops/>
- [5] ——. (2026) What is devsecops? [Online]. Available: <https://aws.amazon.com/what-is/devsecops/>
- [6] GitHub. (2026) Github actions documentation. [Online]. Available: <https://docs.github.com/en/actions>
- [7] HashiCorp. (2026) What is infrastructure as code with terraform? [Online]. Available: <https://developer.hashicorp.com/terraform/tutorials/aws-get-started/infrastructure-as-code>
- [8] ——. (2026) Overview of the core terraform workflow. [Online]. Available: <https://developer.hashicorp.com/terraform/intro/core-workflow>
- [9] Docker, Inc. (2026) What is a container? [Online]. Available: <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/>
- [10] The Kubernetes Authors. (2026) Deployments. [Online]. Available: <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>
- [11] Google Cloud. (2024) Mlops: Continuous delivery and automation pipelines in machine learning. [Online]. Available: <https://docs.cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>
- [12] OpenTelemetry Authors. (2026) Observability primer. [Online]. Available: <https://opentelemetry.io/docs/concepts/observability-primer/>

- [13] Google Cloud. (2026) Gke autopilot overview. [Online]. Available: <https://docs.cloud.google.com/kubernetes-engine/docs/concepts/autopilot-overview>
- [14] Helm Authors. (2026) Charts. [Online]. Available: <https://helm.sh/docs/topics/charts/>
- [15] Iterative, Inc. (2026) Dvc command reference. [Online]. Available: <https://dvc.org/doc/command-reference>
- [16] MLflow Authors. (2026) MLflow tracking. [Online]. Available: <https://mlflow.org/docs/latest/ml/tracking>
- [17] Google Cloud. (2026) Cloud sql overview. [Online]. Available: <https://docs.cloud.google.com/sql/docs/postgres/introduction>
- [18] Redis. (2026) Redis data types. [Online]. Available: <https://redis.io/docs/latest/develop/data-types/>
- [19] Google Cloud. (2026) Cloud monitoring overview. [Online]. Available: <https://docs.cloud.google.com/monitoring/docs/monitoring-overview>
- [20] Prometheus Authors. (2026) Metric types. [Online]. Available: https://prometheus.io/docs/concepts/metric_types/
- [21] Google Cloud. (2026) Managed service for prometheus. [Online]. Available: <https://docs.cloud.google.com/stackdriver/docs/managed-prometheus>
- [22] Scikit-learn Developers. (2026) Tfidfvectorizer and cosine similarity. [Online]. Available: <https://scikit-learn.org/stable/>

JITS INNOVATION LABS

Ý kiến đánh giá:

Hà Nội, ngày tháng năm 20 .

Người hướng dẫn

(Ký, ghi rõ họ tên & dấu công ty)



Ý kiến đánh giá:

Điểm số:

Điểm chữ:

Hà Nội, ngày tháng năm 20 .

Giảng viên đánh giá

(Ký, ghi rõ họ tên)